

Lab 3

Interprocess Communication

Due Mon Oct 5, 11:59 PM CT



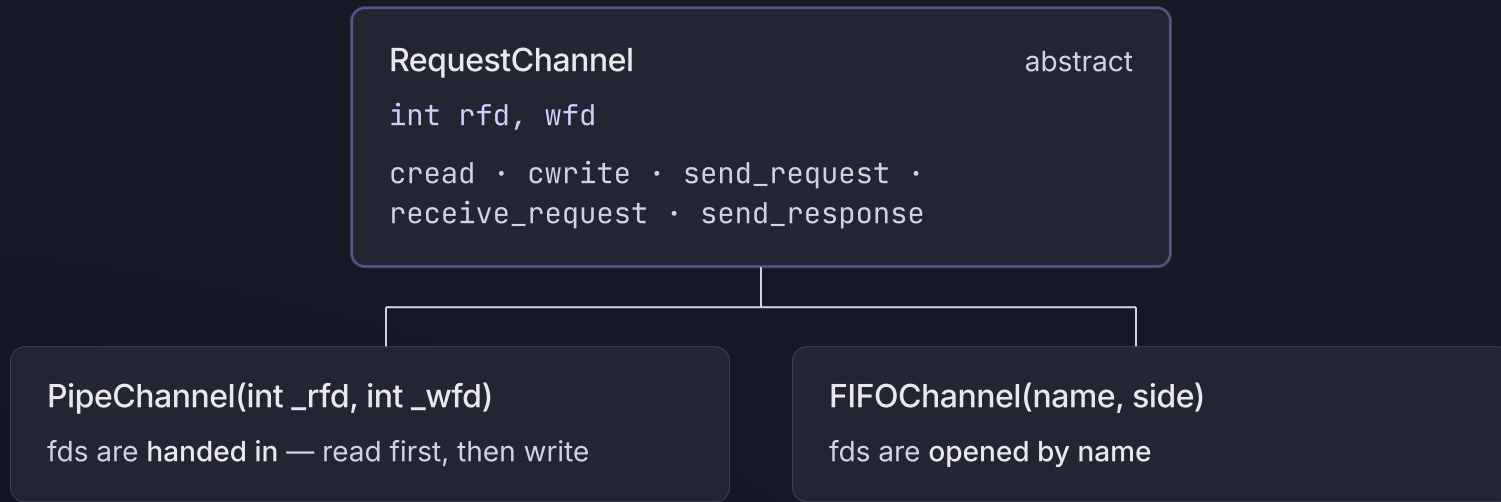
What changed since Lab 2

TOPIC	LAB 2	LAB 3
Channel	Hidden in LLVM IR	Yours, in <code>channel.cpp</code>
Classes	One <code>RequestChannel</code>	<code>AbstractRequestChannel</code> + <code>PipeChannel</code> + <code>FIFOChannel</code>
First prompt	—	Choose IPC type, 1 for PIPE, 2 for FIFO:
Message text	<code>std::string</code>	Fixed char arrays
QUIT	—	Server replies, then exits
Log flag	<code>-f <file></code>	<code>-n <file></code> (<code>-f</code> now means FIFO mode)
Compiler	<code>clang</code>	<code>g++ -std=c++17</code>

A channel is two file descriptors

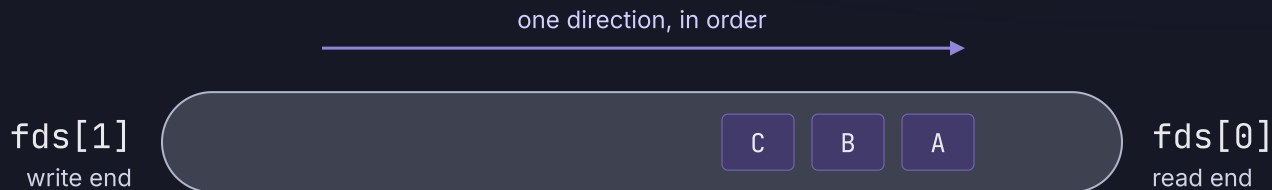
— RULE

A channel is just two file descriptors: `rfd` and `wfd`.



pipe() in one picture

```
int fds[2]; pipe(fds); 0 reads like stdin · 1 writes like stdout
```



One direction

In order

`read()` blocks while empty

1 connection = 2 pipes

3 servers = 6 pipes

fork() copies the fd table, not the pipe

BEFORE

parent

0	stdin
1	stdout
2	stderr
3	pipe[0]
4	pipe[1]

fork()



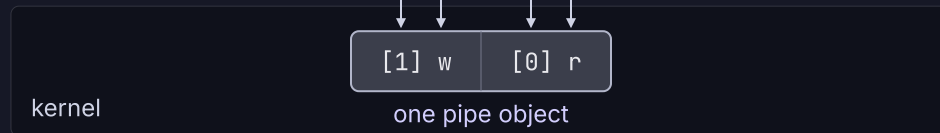
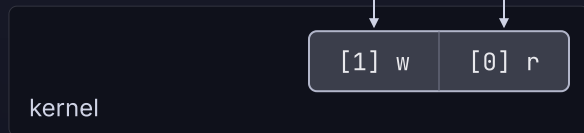
AFTER

parent

0	stdin
1	stdout
2	stderr
3	pipe[0]
4	pipe[1]

child

0	stdin
1	stdout
2	stderr
3	pipe[0]
4	pipe[1]



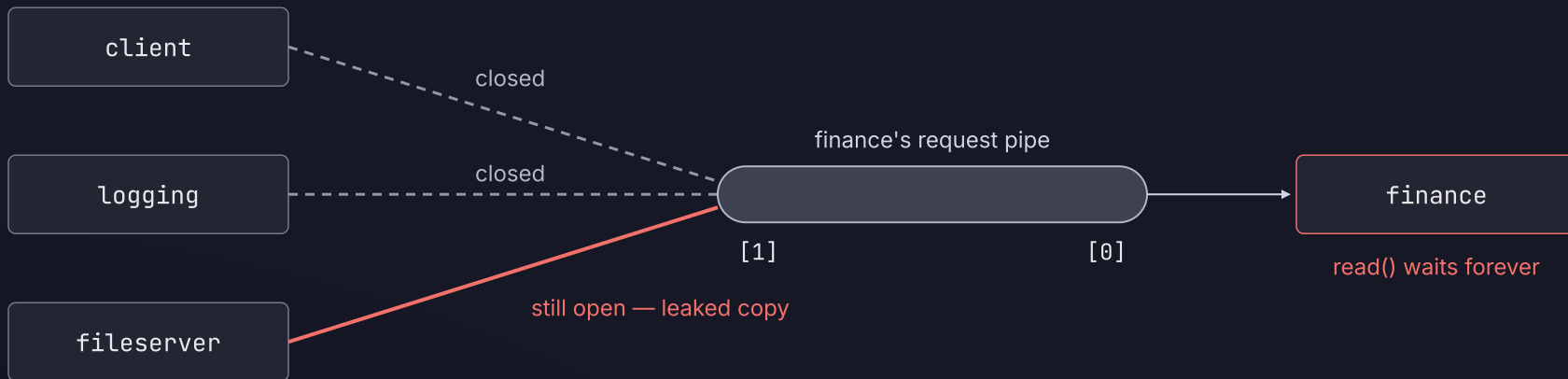
— RULE

Pipes have no name. Create them before `fork()`, and every child inherits all of them.

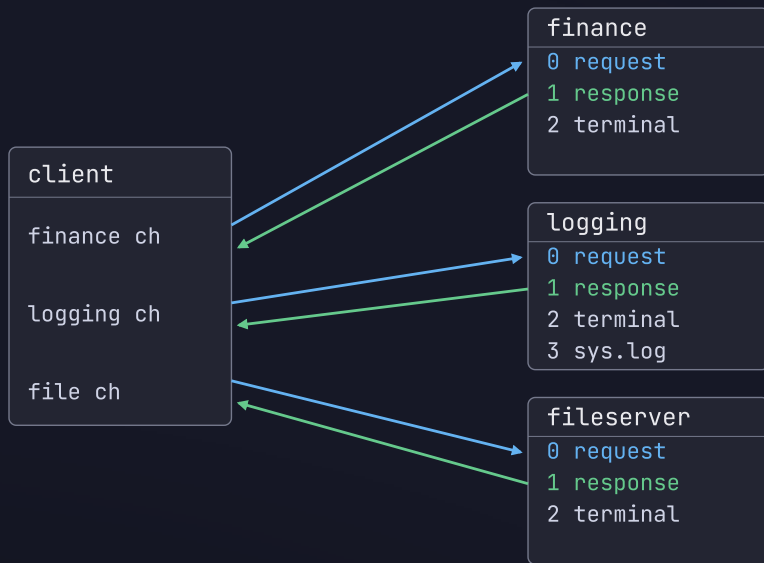
The EOF rule

— RULE

`read()` returns 0 only when every write end is closed, in every process.



Wiring pipe mode: the fd tables, measured



— request pipe → fd 0 — response pipe ← fd 1

✓ CORRECT

```

finance:    0 1 2
logging:    0 1 2 3→sys.log
fileserver: 0 1 2
  
```

⊗ LEAKY — CHILD CLOSES ONLY ITS OWN 4

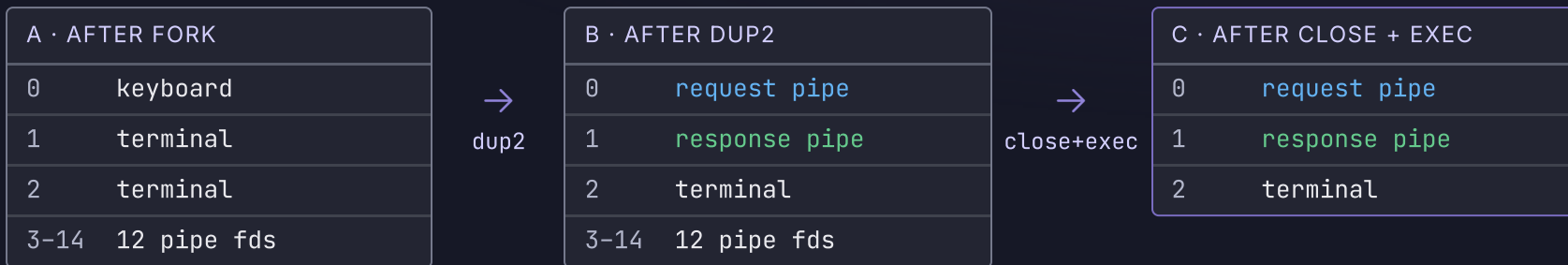
```

finance:    0 1 2 + 8 extra pipe fds
logging:    0 1 2 log + 8 extra
fileserver: 0 1 2 + 8 extra
  
```

— RULE

Each child: dup2 its two ends onto 0 and 1, then close all 12.

dup2 + exec: the server never sees a pipe



Every server starts with one line:

```
new PipeChannel(STDIN_FILENO,
STDOUT_FILENO)
```

⚠ COSTS POINTS -10 LOCALLY

In pipe mode, a server's stdout is the response pipe. Use cerr.

```
cout << "debug: got request" << endl;    // in finance.cpp
...
Deposit successful. New balance: 5.56112e+180
Withdrawal failed:
```

SIGPIPE and the stolen keystroke

⚠ COSTS POINTS -15

SYMPTOM

One answer you type seems ignored. Later, the client quits silently at the first deposit.

CAUSE

Finance's stdin was never redirected. It is forked *before* the log-file prompt, so it reads the keyboard, takes a line, and exits. Nobody reads its request pipe, so the client's write triggers **SIGPIPE**.

```
$ ./client
...           # client vanishes at the deposit
$ echo $?
141           # 128 + 13 = SIGPIPE
```

141 = 128 + signal 13 (SIGPIPE)

FIFOs: pipes with a name

- `mkfifo(name, 0600)` creates a file; any process can `open()` it.
- `open()` blocks until the other end opens too.

<code>fifo_finance1</code>	<code>fifo_finance2</code>
<code>fifo_logging1</code>	<code>fifo_logging2</code>
<code>fifo_file1</code>	<code>fifo_file2</code>

Use exactly `finance`, `logging`, `file`.

EEXIST

expected

Both sides call `mkfifo`, so the second one gets `EEXIST`.

ENOENT

expected

Both destructors `unlink`, so the second one gets `ENOENT`.

The open-order deadlock

✓ SAME ORDER

client		finance
open ...1 (waits)	↔	open ...1
open ...2	↔	open ...2

Both unblock at each step. Ready.

✗ REVERSED ON ONE SIDE

client		finance
open ...1		open ...2
waits for finance		waits for client

Each waits forever. Nothing prints.

— RULE

Open ...1 first on both sides, then ...2, with complementary modes.

Messages are raw bytes

Request

$4 + 4 + 8 + 256 + 1024 = 1296 \text{ B}$



Response

$1 + 7 \text{ pad} + 8 + 1024 + 256 = 1296 \text{ B}$



- One `cwrite(&req, sizeof(Request))` sends it all. A `std::string` would send a heap pointer.
- $1296 < 4096$ (`PIPE_BUF`), so each message arrives whole.
- Anything but the full size = failure. `cread` returning 0 = EOF → `Request(FAILURE)`.

⚠ COSTS POINTS -5

`cread(...) < sizeof(Request): -1` becomes a huge unsigned, so an error looks like success. The build warns (`-Wsign-compare`).

⚠ WON'T COMPILE

`cwrite(&req, ...)` with `const Request& req`: the parameter is `void*`, so cast.

Local vs autograder

`make test · local`

Everything uses **your** channel.

On push · autograder

Unit tests use **your** channel (75 pts).

Client and server sessions use the **reference** channel and counterparts (25 pts).

— RULE

A channel bug costs more locally. A self-consistent mistake passes locally and fails on push.

e.g. a FIFO name misspelled the same way on both sides



— tests your channel — reference channel on push · Client includes a 5-pt kill test

Debug toolkit

```
$ make test                                # autograder checks, /100
$ ls -l /proc/<pid>/fd                      # a server: only 0 1 2 (+ log)
$ pgrep -a -x 'finance|logging|fileserver' # orphans?
$ ls -l fifo_*                             # leftovers? ('p' = FIFO)
$ echo $?                                  # 141 = SIGPIPE
$ make distclean                           # removes stale fifo_* files
```

⚠ NEVER

Never pipe an input file into `./client` or redirect its output unattended. At stdin EOF the starter loops forever printing “Invalid input”.